

Lecture 7: Multiple Access Protocols

MAC Layer: How Nodes Share a Channel

EC 441 - Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Explain the multiple access problem and why it arises on shared broadcast channels
- Describe channel partitioning approaches (TDMA, FDMA) and their efficiency limitations
- Describe taking-turns approaches (polling, token passing) and their tradeoffs
- Derive the throughput of pure ALOHA and slotted ALOHA; identify maximum efficiency
- Explain CSMA and how carrier sensing improves on ALOHA
- Describe CSMA/CD and explain why a minimum frame size is required
- Identify which MAC protocol is used in Ethernet vs. WiFi

The Multiple Access Problem

Broadcast channel: multiple nodes transmit and receive on the **same shared medium**

Examples:

- Coaxial cable segment (classic Ethernet)
- WiFi radio channel
- Satellite uplink used by many ground stations

The problem: if two nodes transmit simultaneously, signals interfere

Collision $\Rightarrow$ both frames are garbled and lost

A **multiple access (MAC) protocol** is a distributed algorithm that coordinates who transmits when.

Channel Directionality

Three modes of communication on a link:

Mode	Description	Examples
Simplex	One direction only	Broadcast radio, cable TV
Half-duplex	Both directions, not simultaneously	Classic Ethernet, WiFi, walkie-talkie
Full-duplex	Both directions simultaneously	Telephone, switched Ethernet

Key point: The MAC problem is a **half-duplex** problem.

On a shared half-duplex channel, only one node can transmit at a time — hence the need for coordination.

On a **full-duplex** point-to-point link, each direction is independent: no collisions, no MAC protocol needed.

What We Need From a MAC Protocol

Ideal MAC protocol for channel of rate R :

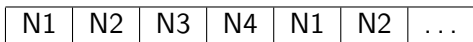
- 1 When **one node** has data: it gets full rate R
- 2 When M **nodes** have data: each gets roughly R/M on average
- 3 Fully **decentralized** (no central coordinator, no single point of failure)
- 4 **Simple** to implement

Three families of solutions:

Channel partitioning	Divide channel; give each node a piece
Taking turns	Nodes take turns explicitly
Random access	Transmit freely; detect/recover from collisions

TDMA — Time Division Multiple Access

Idea: Divide time into frames; each of N nodes gets one fixed slot per frame



Properties:

- Each node gets rate R/N — guaranteed, always
- **No collisions**
- **Efficiency problem:** if only one node has data, channel is $1/N$ utilized
- Idle slots are wasted — no other node can use them

Used in: GSM cellular, satellite channels, some industrial networks

FDMA and CDMA

FDMA — Frequency Division Multiple Access

- Divide bandwidth W into N frequency bands; each node gets one band permanently
- Same efficiency problem as TDMA: wasted bands when nodes are idle
- Used in: FM radio, cable TV, traditional analog cellular

CDMA — Code Division Multiple Access (brief mention)

- Each node uses an orthogonal **spreading code**; all nodes share full bandwidth simultaneously
- Receiver separates signals using the code
- Used in: 3G cellular (WCDMA), GPS
- Not covered in depth in this course

Summary critique: Channel partitioning works well under **heavy, symmetric** load. Most network traffic is **bursty** — partitioning wastes capacity.

Taking Turns: Polling

Idea: A coordinator node polls each node in round-robin order, granting permission to transmit

Properties:

- No collisions
- Master can enforce fairness
- **Polling overhead:** one poll message per node per round, even if idle
- **Master is a single point of failure**
- Latency even when most nodes have nothing to send

Used in: Bluetooth (primary/secondary device model), industrial fieldbus protocols

Taking Turns: Token Passing

Idea: A special **token** frame circulates around a logical ring. A node may transmit only while holding the token; then passes it on.

Node A $\rightarrow$ Node B $\rightarrow$ Node C $\rightarrow$ Node D $\rightarrow$ Node A $\rightarrow \dots$

(token circulates; holder may transmit)

Advantages:

- No collisions
- Predictable worst-case latency — good for real-time/industrial systems

Problems:

- Token loss requires recovery protocol
- One failed node can break the ring
- Overhead when few nodes have data

Historically: Token Ring (IEEE 802.5), FDDI — switched Ethernet dominates today

Random Access: The ALOHA Idea

Key insight: Don't coordinate in advance — just transmit. Handle collisions after the fact.

Historical origin: ALOHAnet (University of Hawaii, 1970)

- Multiple ground stations sharing a single satellite uplink
- No centralized controller possible
- Simple, fully distributed protocol needed

Wikipedia: [Wikipedia page]

Pure ALOHA rule:

*If you have a frame, transmit it immediately.
If a collision occurs, wait a random time and retransmit.*

Pure ALOHA: Collision Window

When can two frames collide?

Let frame transmission time = T (time to send one frame).

A frame sent at time t occupies the channel during $[t, t + T]$.

It can collide with any frame whose transmission begins during $[t - T, t + T]$:

$$\text{Collision window} = 2T$$

Intuition: A late-starting frame can collide at the tail of our frame; an early-starting frame's tail can collide with our start.

⇒ Pure ALOHA has the **largest possible collision window**. This drives low efficiency.

Pure ALOHA: Throughput Analysis

Let G = offered load (average number of transmission attempts per frame-time T).

Assume Poisson arrivals. A transmission succeeds if **no other node transmits** during the collision window $2T$:

$$P(\text{success}) = e^{-2G}$$

Throughput (successful transmissions per frame-time):

$$S = G e^{-2G}$$

Maximize: differentiate and set to zero $\Rightarrow G = 1/2$

$$S_{\max} = \frac{1}{2} \cdot e^{-1} = \frac{1}{2e} \approx 0.184$$

Pure ALOHA maximum efficiency: $\approx 18\%$

Slotted ALOHA: Halving the Collision Window

Key idea: Synchronize all nodes to a common slot clock.

Rule: Transmissions begin *only at slot boundaries*. If collision: retransmit in a future slot with probability p (independently each slot).

Effect on collision window:

- Two frames can only collide if they begin in the **same slot**
- Collision window shrinks from $2T$ to T

Throughput analysis with offered load G attempts per slot:

$$P(\text{success}) = e^{-G} \quad \Rightarrow \quad \boxed{S = G e^{-G}}$$

Maximize: $G = 1 \Rightarrow$

$$S_{\max} = e^{-1} = \frac{1}{e} \approx 0.368$$

Slotted ALOHA maximum efficiency: $\approx 37\%$ — exactly double pure ALOHA

Slotted ALOHA: What Happens at Peak Load?

At $G = 1$ (the efficiency-maximizing load):

Slot outcome	Probability	Note
Successful frame	$1/e \approx 37\%$	Exactly one transmission
Empty slot	$1/e \approx 37\%$	No transmissions
Collision	$1 - 2/e \approx 26\%$	Two or more transmissions

Practical issue: Requires slot synchronization across all nodes — non-trivial in a distributed system.

Can we do better without synchronization? Yes — with **carrier sensing**.

ALOHA Summary and Motivation for CSMA

Protocol	Max Efficiency	Collision Window	Sync?
Pure ALOHA	$\approx 18\%$	$2T$	No
Slotted ALOHA	$\approx 37\%$	T	Yes
CSMA (next)	Much better	$\tau \ll T$	No

ALOHA weakness: nodes transmit without knowing what others are doing.

Key observation: Signals propagate at roughly 2×10^8 m/s. On a LAN, propagation delay τ is **much smaller** than frame transmission time T .

$\Rightarrow$ Before transmitting, we can **listen** to see if the channel is in use.

CSMA — Carrier Sense Multiple Access

Carrier sense: Before transmitting, listen to the channel.

- Channel **idle** $\Rightarrow$ transmit
- Channel **busy** $\Rightarrow$ defer

Why does this help? Collisions only occur when two nodes *both* sense idle within one propagation delay τ of each other. Since $\tau \ll T$, efficiency is much higher than ALOHA.

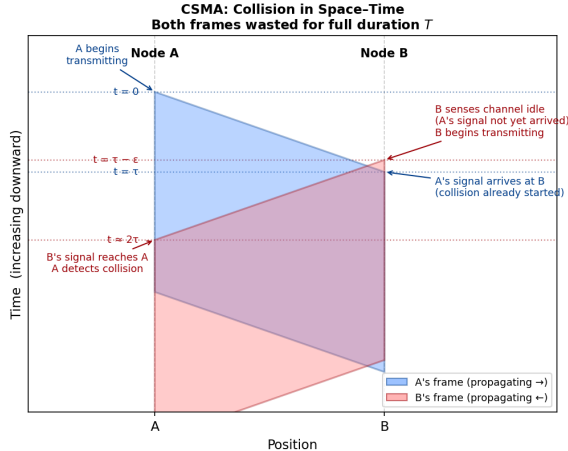
CSMA variants:

- **1-persistent:** if idle, transmit immediately; if busy, wait then transmit immediately when idle
 - Problem: all waiting nodes transmit at once when channel clears $\Rightarrow$ guaranteed collision
- **Non-persistent:** if busy, wait a *random* time, then sense again
- **p -persistent:** if idle, transmit with probability p , defer with probability $1 - p$

Wikipedia: [Wikipedia page]

CSMA: Collisions Still Happen

Problem: Two nodes can both sense the channel idle and start transmitting simultaneously — within one propagation delay τ of each other.



Collision detection (CD): While transmitting, **listen to the channel simultaneously**.

If what you hear $\neq$ what you're sending $\Rightarrow$ collision is occurring $\Rightarrow$ **abort immediately**.

Key improvement: Instead of wasting T on a collided frame, abort after at most 2τ (round-trip propagation delay). Since $\tau \ll T$, waste is minimal.

This is the basis of **classic (shared) Ethernet** — IEEE 802.3.

Wikipedia: [Wikipedia page]

CSMA/CD Algorithm

- ① If channel idle $\Rightarrow$ transmit frame (1-persistent)
- ② If channel busy $\Rightarrow$ wait until idle, then transmit
- ③ While transmitting, **monitor the channel**
- ④ If collision detected:
 - ① Send **jam signal** (48 bits) to ensure all nodes detect the collision
 - ② **Abort** transmission
 - ③ Enter **binary exponential backoff**
- ⑤ After backoff delay, return to step 1

Jam signal: a short burst that reinforces the collision signal so every node on the segment is certain a collision occurred.

Binary Exponential Backoff

After the k -th collision on a frame, choose a random wait from:

$$\{0, 1, 2, \dots, 2^{\min(k,10)} - 1\} \quad \text{slot times}$$

Collision #	Backoff range	Max wait
1	$\{0, 1\}$	2 slots
2	$\{0, 1, 2, 3\}$	4 slots
3	$\{0, \dots, 7\}$	8 slots
$\vdots$	$\vdots$	$\vdots$
10+	$\{0, \dots, 1023\}$	1024 slots
16	Give up; report error to upper layer	

Why exponential? Adapts to channel load:

- Light load $\Rightarrow$ short backoffs $\Rightarrow$ low latency
- Heavy load $\Rightarrow$ long backoffs $\Rightarrow$ nodes spread out, fewer future collisions

Minimum Frame Size: The Constraint

Problem: A node must still be *transmitting* when it hears a collision.

If a node finishes transmitting before the collision signal returns, it has no way to know a collision occurred.

Required condition: Transmission time $T \geq$ round-trip propagation delay 2τ

$$T = \frac{L}{R} \geq 2\tau \quad \Rightarrow \quad \boxed{L \geq 2\tau R}$$

Classic Ethernet example: $R = 10$ Mb/s, max segment = 2500 m

$$\tau \approx 25 \mu\text{s} \quad (\text{at } 2 \times 10^8 \text{ m/s})$$

$$L_{\min} = 2 \times 25 \mu\text{s} \times 10^7 \text{ bps} = 500 \text{ bits} = \mathbf{64} \text{ bytes}$$

The 64-byte minimum Ethernet frame size comes directly from physics.

CSMA/CD Efficiency and the End of Shared Ethernet

CSMA/CD efficiency (for large frames, small $a = \tau/T$):

$$\text{Efficiency} = \frac{1}{1 + 5a}$$

As $a \rightarrow 0$ (frame time $\gg$ propagation delay) $\Rightarrow$ Efficiency $\rightarrow 1$.

End of shared Ethernet:

Modern Ethernet uses **switches** with full-duplex point-to-point links.

- Each device connects to exactly one switch port
- Only two nodes share a link: the device and the switch
- No contention $\Rightarrow$ **collisions are impossible**

CSMA/CD is in the IEEE 802.3 standard but is **never exercised** in modern networks. The protocol for switched Ethernet: “transmit whenever you want.”

(More on Ethernet switching in Lecture 8.)

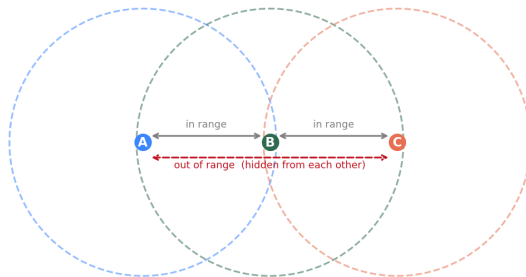
Why Collision Detection Fails in Wireless

CSMA/CD requires: a transmitting node can simultaneously hear what's on the channel.

In wireless, this fails for two reasons:

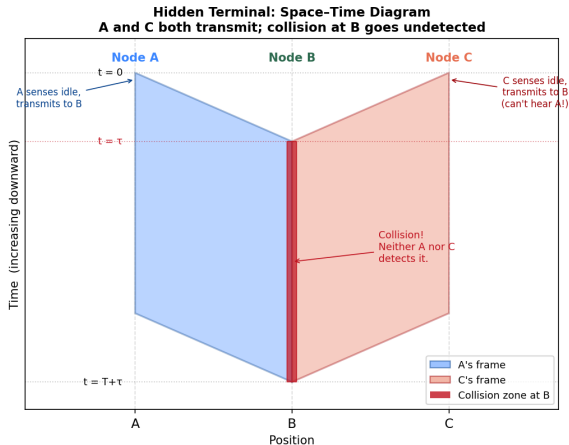
- 1. Self-interference:** The transmitter's own signal is orders of magnitude stronger than any received signal. The radio cannot hear others while transmitting.
- 2. Hidden terminal problem:**

Hidden Terminal Problem: Spatial Layout



Why CD Doesn't Work in Wireless (cont.)

Hidden terminal: space-time view



⇒ Wireless needs a different strategy: **collision avoidance**

CSMA/CA Preview (WiFi / 802.11)

Collision avoidance: Try to prevent collisions rather than detect them.

Key ideas in CSMA/CA:

- Use random backoff **before** transmitting, not just after collision
- Optional **RTS/CTS** (Request to Send / Clear to Send) exchange to reserve the channel before the data frame
- **ACK every frame**; no ACK $\Rightarrow$ assume collision, retransmit

RTS/CTS idea:

- 1 Sender broadcasts short RTS frame (“I want to send to B for X ms”)
- 2 B responds with CTS (“OK, everyone else defer for X ms”)
- 3 All nodes in range hear RTS or CTS $\Rightarrow$ defer (solves hidden terminal)
- 4 Sender transmits data; B ACKs

Details in Lecture 9 (802.11 Wireless Networks).

MAC Protocols: Summary

Family	Protocol	Used In	Key Mechanism
Partitioning	TDMA	GSM, satellite	Fixed time slots
Partitioning	FDMA	FM radio, cable	Fixed frequency bands
Taking turns	Token Ring	Legacy LANs	Circulating token
Random access	Pure ALOHA	Packet radio	Transmit freely
Random access	Slotted ALOHA	Satellite, LTE	Slot-synchronized
Random access	CSMA/CD	Classic Ethernet	Sense + detect + backoff
Random access	CSMA/CA	WiFi (802.11)	Sense + avoid + ACK

Key Numbers to Remember

Result	Value
Pure ALOHA max efficiency	$S_{\max} = 1/(2e) \approx 18\%$ at $G = 1/2$
Slotted ALOHA max efficiency	$S_{\max} = 1/e \approx 37\%$ at $G = 1$
Slotted vs. pure	Slotted is exactly $2\times$ better
Ethernet min frame	64 bytes (from $L \geq 2\tau R$)
Classic Ethernet round-trip delay	$2\tau \approx 50 \mu\text{s}$ (2500m segment)

The big picture:

- Channel partitioning: wastes capacity under bursty load
- ALOHA: simple but low efficiency
- CSMA + collision detection/avoidance: near-ideal under typical load
- Modern Ethernet: switched, full-duplex — MAC problem mostly goes away

Lecture 8: Ethernet — Addressing, Switching, ARP

- Ethernet frame structure and field meanings
- MAC addresses: format, assignment, broadcast
- ARP: how a node discovers the MAC address for a given IP address
- Ethernet switches: forwarding tables, self-learning algorithm
- Multi-switch topologies and worked examples

Connection to today: Every Ethernet frame uses CRC-32 for error detection (Lecture 6) and relies on CSMA/CD for channel access on legacy segments (today). Switched Ethernet replaces CSMA/CD but keeps the frame format and MAC addressing.